Bridging the Gap Between the Mano Register-Transfer Model and FPGA Realization Through a Synthesisable VHDL-93 Architecture

Musthofa Joko Anggoro
Faculty of Computer Science
Universitas Indonesia
Depok, Indonesia
Email: musthofa.joko@ui.ac.id

Petrus Mursanto
Faculty of Computer Science
Universitas Indonesia
Depok, Indonesia
Email: santo@cs.ui.ac.id

***Abstract*—The register-transfer model proposed by Mano defines a complete conceptual computer architecture at the register-transfer level, specifying named registers and a hardwired control unit, and serves as a standard pedagogical reference in undergraduate computer architecture courses. Several engineering constraints are left undefined when a synthesisable field-programmable gate array (FPGA) realisation is attempted: register organisation, instruction encoding, the control mechanism, memory partitioning, and the I/O interface. A systematic gap analysis between the Mano model and the requirements of a synthesisable FPGA implementation was conducted, and each identified constraint was mapped to a concrete design decision. A complete, hierarchically-structured 8-bit datapath was subsequently designed and implemented in VHDL-93 (IEEE 1076-1993), targeting the Xilinx Spartan-3AN starter board. Verification was performed at each tier of a five-tier component hierarchy through functional simulation testbenches run under ISim and GHDL and through board-level testing on the target device, including a board-level demonstration with LCD display output matching the simulated microoperation sequence at each step. Post-place-and-route static timing analysis confirmed a minimum clock period of 17.051 ns (maximum frequency 58.648 MHz), satisfying the 50 MHz board-oscillator constraint. All hardware modules passed testbench verification; board-level testing on the physical device additionally surfaced one defect that the existing testbench suite had missed, which was corrected and closed with two added regression testbenches. These results confirm that the Mano register-transfer model can be implemented, to the point a synthesis tool accepts and routes the design, as a synthesisable VHDL-93 design on an FPGA while preserving the single-cycle microoperation contract.**

***Index Terms*—datapath, FPGA, Mano register-transfer model, Spartan-3AN, VHDL**

I. Introduction

Mano [1] defines a register-transfer model of a stored-program computer as a fixed set of named registers driven by a hardwired *control unit*, in which every timing state of every instruction is specified by a conditional *register-transfer statement* that asserts data movement between registers. This level of formalism has made the model a standard reference in undergraduate computer architecture instruction. The model is complete at the conceptual level, yet its register organisation, instruction encoding, control mechanism, memory model, and I/O interface are each specified abstractly, with no direct correspondence to synthesisable *VHDL* constructs or to the physical resources of a specific target device. Realising the model on a physical *field-programmable gate array* (FPGA) board additionally makes each register-transfer step directly observable as a concrete signal change on board hardware, rather than symbolic or simulated.

This paper makes three contributions. First, a systematic *gap analysis* compares the requirements of a general-purpose stored-program computer against the capabilities of the Mano register-transfer model and maps each identified constraint to a concrete design decision. Second, a hierarchically-structured 8-bit *datapath*, a contraction from the 16-bit word width of the original Mano specification, is designed and implemented in *VHDL-93* (IEEE 1076-1993), preserving the single-cycle *microoperation* contract of the original model in its execute phase. Third, the resulting design is verified through functional simulation and demonstrated on a Xilinx Spartan-3AN starter board, confirming that the synthesised behaviour matches the register-transfer specification at every step. The remainder of this paper is organised as follows: Section II positions this work against prior FPGA-based computer-architecture education, Section III reviews the Mano datapath and target FPGA platform, Section IV describes the datapath design, Section V presents the gap analysis and its FPGA implementation, Section VI reports verification and board-demonstration results, and Section VII concludes.

II. Related Work

Prior work on realising computer-architecture teaching tools on FPGAs generally designs a new or reduced instruction set expressly for classroom use, rather than targeting an existing published model. Holland et al. [2] implement a single-cycle processor built from a reduced eight-instruction subset of the MIPS instruction set on a Xilinx Virtex FPGA, paired with a parallel-port debugging tool that gives students software-level control and observability of internal processor state. Guštin and Bulić [3] design MOVE, a 16-bit *move-only* RISC instruction set with a single operation code, and realise it on

a Xilinx Spartan-II device, reporting resource utilisation for alternative register-file encodings. Cifredo-Chacón et al. [4] report a bottom-up methodology in which first-year students build DidaComp, a 4-bit, four-instruction computer, from individual VHDL blocks on a low-cost FPGA board, and back the approach with course-outcome and survey data.

This paper differs from all three in taking an existing, published register-transfer specification, Mano's model [1], as the object of study rather than designing a new instructional instruction set; no prior synthesisable FPGA realisation of Mano's specific model was found in the reviewed literature. Device families differ too much for a controlled comparison, but this design's 14% LUT, 5% flip-flop footprint at 58.648 MHz (Section V) is reported alongside Holland et al.'s pipelined 29% LUT at 1.4 MHz and Guštin and Bulić's 29% LUT, 39% slice MOVE design only for context, in part reflecting a smaller register file; Guštin and Bulić's register-file encoding trade-off parallels the register-organisation gap in Table I. Holland et al.'s host-PC debugging tool and this paper's standalone front panel both address observability by different means; unlike Cifredo-Chacón et al.'s course-outcome data, this paper reports none, leaving that study to Section VII-A.

III. Background

The Mano Basic Computer [1] is a 16-bit stored-program machine whose eight named *processor registers* interface to a single common bus under the direction of a hardwired *control unit*; a *sequence counter* steps the control unit through a fixed sequence of *timing states* for each instruction, and each state asserts a register-transfer statement that moves data between registers and the arithmetic-logic unit. The 8-bit datapath presented in this paper builds on the generic register-file reference architecture described by Mano and Kime [5], in which a multiplexer-controlled register file supplies two source operands to a *function unit* composed of an ALU and a shifter, and a write-back multiplexer routes the function-unit result or an external data value back to the register file while the ALU and zero-detect logic generate the status flags consumed by the surrounding control logic.

The design targets the *Xilinx Spartan-3AN XC3S700AN* [6], a field-programmable gate array that provides 5,888 configurable-logic-block slices (11,776 four-input look-up tables and 11,776 flip-flops in total) and 20 dedicated *block RAM* tiles of 18 Kbits each [7], of which two tiles are used to implement the microoperation-program memory and the data memory of the datapath described in Section IV.

IV. Datapath Design and Methodology

The datapath generalises the single-bus Mano reference architecture to a *three-bus architecture* in which separate buses carry the two source operands and the result, allowing a full register-to-register operation to complete in a single clock cycle. Bus A carries the first source operand from the register-file output port A; Bus B carries either the second source operand from register-file port B or an 8-bit CONSTANT literal, selected by MUX B; and Bus D carries the write-back value to the destination register, selected by MUX D between the *function unit* output and an external DATA_IN literal.

Fig. 1 presents the resulting RTL schematic, synthesised by Xilinx ISE XST.

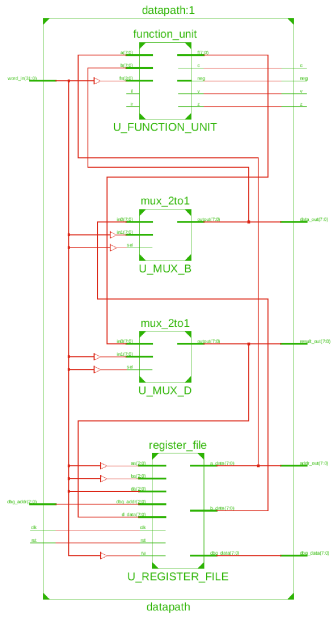

Fig. 1. RTL schematic of the datapath module as synthesised by Xilinx ISE XST, showing the register_file, MUX B, function_unit, and MUX D.
Source: Xilinx ISE Design Suite [8].

The synthesised connectivity confirms the intended three-bus structure with no extraneous logic: register_file drives Bus A and Bus B directly into MUX B and the function unit, and MUX D returns the sole write-back path into the register file. The function unit itself combines an ALU and a shifter, with MUX F selecting between their outputs according to the function-select field of the control word. At the top level, the fpga_top entity integrates four subsystems: input_ctrl, which debounces push-buttons and decodes PS/2 keyboard input; memory_ctrl, which holds the two block-RAM-backed memory arrays; datapath_system, which combines the sequencer and the three-bus datapath described above; and ui_ctrl, which implements the mode-selection logic and the LCD display interface.

Fig. 2 shows the resulting top-level RTL schematic, synthesised by Xilinx ISE XST.

The schematic confirms the four subsystems connect through a single top-level port map, with no stray signals bypassing datapath_system.

Each VHDL entity is written in one of three modelling styles, assigned per module role rather than uniformly across the design. Leaf-level combinational primitives and the register file use the *behavioural* style: a single clocked process guarded by if rising_edge(clk) is the VHDL-93 idiomatic form for D-flip-flop inference, recognised by XST without additional synthesis attributes [9]. Listing 1 shows this template ap-

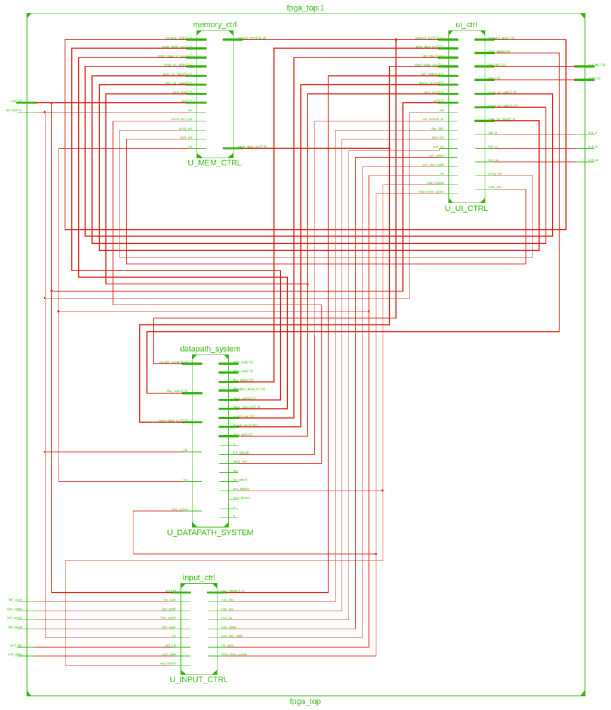

Fig. 2. Top-level RTL schematic of the fpga_top entity generated by Xilinx ISE XST synthesis, showing the four subsystem blocks (input_ctrl, memory_ctrl, datapath_system, ui_ctrl) and their top-level interconnections. **Source:** Xilinx ISE Design Suite [8].

plied to reg_n; each instance maps to exactly eight flip-flops with no combinational overhead, an exact match between the generic parameter and the synthesised count. Integration layers (datapath, datapath_system, fpga_top) use the *structural* style, composed only of component instantiations, so the full connectivity is readable from the source without a synthesised schematic; combinational glue such as the control-word decoder uses the *RTL dataflow* style of concurrent signal assignments instead. The full design's 638 synthesised flip-flops are consistent with the per-module counts expected from this template.

Listing 1. Behavioural synchronous template applied to reg_n.

```
architecture rtl of reg_n is
  signal q_reg : STD_LOGIC_VECTOR(N-1 downto 0) :=
      (others => '0');
begin
  process (clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        q_reg <= (others => '0');
      elsif load = '1' then
        q_reg <= d;
      end if;
    end if;
  end process;
  q <= q_reg;
end architecture rtl;
```

This template is instantiated unmodified across every clocked register in the design, keeping the flip-flop count directly traceable to the generic parameter at each call site. The system was developed using a *bottom-up hierarchical design* strategy, in which every hardware entity is verified independently before being instantiated inside a higher-level module, reducing the effort needed to isolate faults discovered during integration [9]. The design was verified through an iterative write-simulate-synthesise-route-board cycle across seven system-level iterations, comprising 22 feedback-driven fixes: eight simulation events in which testbench stimulus exposed an RTL defect, two schematic-clarity refactors, six place-and-route/timing fixes, and six board-level fixes revealed only once the design ran on physical hardware. As one representative example of the place-and-route category, an early bottleneck arose because microop_memory was initially inferred as distributed RAM, extending each place-and-route run to approximately 18 minutes; switching the array to block-RAM inference resolved the bottleneck and restored a normal run time.

V. Gap Analysis and FPGA Implementation

Comparing the functional requirements of a general-purpose stored-program computer against the capabilities documented for the Mano register-transfer model [1] reveals five engineering gaps, each of which is resolved by a specific design decision in this work. Table I summarises each identified gap and the corresponding design decision, spanning register organisation, instruction-encoding width, control mechanism, memory model, and I/O interface.

The most significant of the five gaps concerns the instruction-encoding width (independent of this implementation's 8-bit datapath width): the 16-bit Mano instruction word provides no space for a destination address, two source addresses, an immediate constant, and multiplexer control bits simultaneously, so the control word is widened to 32 bits. Fig. 3 shows the resulting nine-field layout: DA[31:29] addresses the destination register, RW[28] enables the write-back, AA[27:25] and BA[24:22] address the two source operands, CONSTANT[21:14] carries an 8-bit immediate or branch target, MB[13] selects the Bus B source, FS[12:9] selects one of 16 function-unit operations, MD[8] selects the write-back source, and DATA_IN[7:0] carries an external data byte or branch condition code.

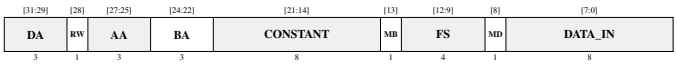

Fig. 3. Bit-field layout of the 32-bit *micro-operation* control word.

The nine fields sum exactly to 32 bits with no unused or overlapping bit positions, confirming that the widened control word accommodates every required field within a single machine word. Closely tied to the encoding gap, the eight dedicated Mano processor registers are replaced by a uniform register file of eight general-purpose registers, R0–R7, each addressable by the 3-bit fields of the control word rather than by a fixed functional role [5]. The single Mano

TABLE I
SUMMARY OF IDENTIFIED GAPS BETWEEN THE MANO REGISTER-TRANSFER MODEL AND A FULLY SYNTHESISABLE, BOARD-INTEGRATED IMPLEMENTATION, WITH THE CORRESPONDING DESIGN DECISION FOR EACH GAP.

Identified Gap	**Design Decision**
Fixed eight-processor-register organisation with dedicated per-register functions; no general-purpose source/destination addressing.	8-register file (R0–R7), each selected by a 3-bit control-word field (DA, AA, BA).
16-bit instruction word cannot encode a destination address, two source addresses, an 8-bit constant, and multiplexer controls simultaneously.	32-bit control word (Fig. 3) with nine non-overlapping fields.
Hardwired combinational control cannot be reprogrammed without redesigning the logic network.	*Microoperation sequencer* reading a 256-word block-RAM program memory via a step counter.
Single unified memory risks a program/data write hazard and does not map to the Spartan-3AN's separate block-RAM resources.	Two block-RAM arrays: microop_memory (256 × 32-bit, program) and data_memory (256 × 8-bit, data).
No physical I/O specification for INPR/OUTR.	HD44780 LCD interface, debounced push-button/PS/2 input, slide-switch mode selection.

memory is likewise partitioned into two independent block-RAM-backed arrays, microop_memory (256 × 32-bit) for the program and data_memory (256 × 8-bit) for operand data, eliminating the hazard of a running program overwriting its own control words.

Post-place-and-route synthesis on the Xilinx Spartan-3AN XC3S700AN device [7] occupies 1,675 of 11,776 available look-up tables (14%), 638 of 11,776 flip-flops (5%), and two of 20 block RAMs (10%). Static timing analysis, run at the worst-case commercial corner (speed grade -4, $85\,^{\circ}$C junction temperature, 1.14 V supply), confirms zero timing violations against the 50 MHz board-oscillator constraint [6], with a minimum achievable clock period of 17.051 ns (58.648 MHz maximum frequency) and a setup slack of 2.949 ns [10]. Per the post-route timing report, the critical combinational path (nine logic levels, 16.946 ns data-path delay) runs from U_MEM_CTRL/U_MICROOP_MEMORY/Mram_mem, the microop_memory block-RAM output, through the register-file B-port multiplexer and the ripple-carry adder of the arithmetic circuit, and ends at U_UI_CTRL/U_LCD/disp_buf_18_1, a display-buffer flip-flop inside the LCD controller [10]. This exceeds any register-to-register path in the core datapath: the 58.648 MHz ceiling is set by the LCD and input-synchroniser paths, the only ones enumerated in the post-route report, not the register-file-to-ALU-to-register-file cycle, whose own worst-case delay is necessarily below 16.946 ns, though not reported separately.

The parallel_adder ripple-carry structure is inferred by the Xilinx XST synthesis engine directly onto the dedicated MUXCY and XORCY carry-chain primitives available in each Spartan-3AN slice, consuming eight MUXCY and eight XORCY stages with zero look-up tables spent on carry propagation itself [11]. No carry-lookahead variant was synthesised for comparison, so the trade-off is not measured directly here. The alu timing report does, however, show that the ripple chain adds negligible aggregate delay relative to the 20 ns period budget at 50 MHz: the first carry cell resolves through MUXCY:S→O at 0.632 ns and each subsequent cell adds only 0.065 ns through MUXCY:CI→O. The overflow flag is computed from the sign bits already present at the arithmetic-circuit boundary, $\overline{A_{n-1} \oplus Y_{n-1}} \cdot (A_{n-1} \oplus \mathrm{Sum}_{n-1})$, synthesising to two one-bit xor2 primitives.

VI. RESULTS AND DISCUSSION

The verification campaign follows a *bottom-up, simulation-first* strategy in which each component is verified in isolation before being composed into a larger subsystem, organised into five verification tiers from leaf-level primitives up to full system integration; a total of 25 self-checking testbenches were written and executed under both the Xilinx ISE Simulator and the GHDL regression runner, each checked against a truth table, timing diagram, or expected register-file state per microprogram step, and all 25 pass with zero *failure*-severity assertions. Table II breaks the campaign down by tier.

At the datapath tier, the datapath_gcd_tb testbench exercises the *subtractive Euclidean* greatest-common-divisor algorithm for the operand pair $(45, 81)$, whose result is 9. The two operands are sourced from data_memory through two LOAD microoperations rather than embedded as immediate constants, exercising the memory-read boundary of the datapath as part of the test; after the subtractive loop converges, a final register readback confirms $\gcd(45, 81) = 9$ for this operand pair.

The synthesised bitstream was programmed onto the Spartan-3AN starter board through the JTAG interface using ISE iMPACT [10]. A 50-step demonstration microprogram was executed on the physical board, and all 50 steps produced LED and LCD output matching the simulation record, with no glitch or metastability failure observed during repeated manual step-and-reset cycles on the target device [6]. Fig. 4 shows the board mid-demonstration, with the LED bank and LCD module displaying the register and status output described above.

TABLE II
VERIFICATION CAMPAIGN BY TIER, ANCHORED AT SUCCESSIVE LEVELS OF THE DESIGN HIERARCHY.
SOURCE: AUTHOR, CONDENSED FROM THE 25-TESTBENCH CAMPAIGN.

Tier	TBs	Representative DUTs
Unit-level	4	reg_n, mux_2to1, shifter, btn_debounce
Function-unit	6	parallel_adder, arithmetic_circuit, logic_circuit, alu, function_unit
Storage subsystem	3	register_file, data_memory, microop_memory
Datapath	6	datapath (decode, timing, GCD, Fibonacci, full-cycle)
System integration	6	datapath_system, ps2_keyboard, fpga_top, ENTRY RAM path

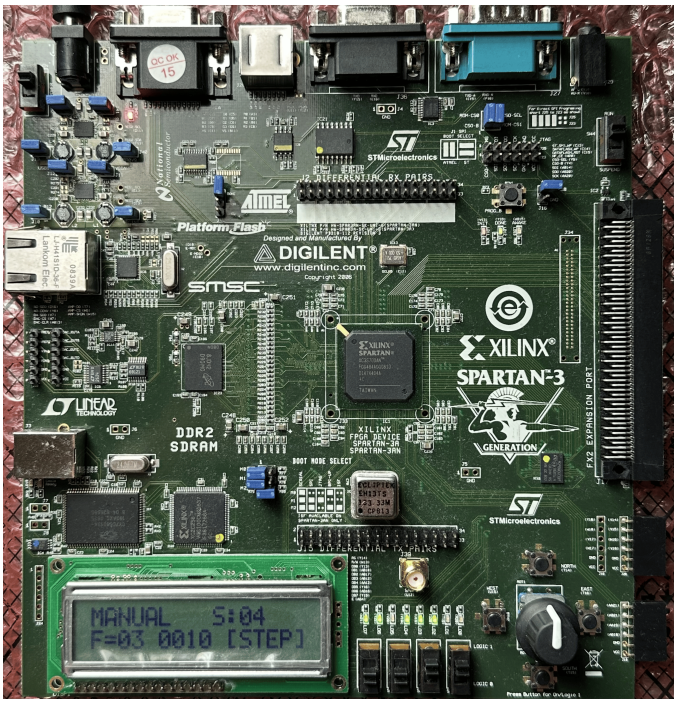

Fig. 4. Spartan-3AN board during the demonstration microprogram. The LED[7:0] bank (bottom centre) shows the datapath result register after the last executed step; the LCD module (bottom left) shows the step counter and the status flag register.
Source: Student photograph, Spartan-3AN Starter Kit board [6].

The displayed LED and LCD values at this step match the corresponding simulation waveform record exactly, confirming that the board-observed register and status output tracks the register-transfer specification rather than diverging from it under physical timing conditions.

A hardware bring-up session on the physical board revealed one defect that no pre-existing testbench had caught: in the ENTRY RAM front-panel mode, a committed byte landed at Mem[N+1] rather than at the user-selected address Mem[N]. The root cause was structural: browse_entry_fsm asserted the write strobe on the same edge that auto-incremented its address cursor, and memory_ctrl drove the data_memory address port directly from that already-incremented cursor rather than the user-selected address; the symmetric ENTRY PROG path was unaffected because it already latched its destination address before the cursor advanced. No existing testbench exercised the front-panel commit path end to end, so this was a coverage gap, not a limit of simulation itself. The fix latches the pre-increment address before the cursor advances, matching the ENTRY PROG pattern, and is locked in by two regression testbenches added to cover the front-panel commit path.

A. Pedagogical Application

The board realisation gives the direct-observability motivation introduced in Section I a concrete outlet: each register-transfer step of the Mano model becomes a directly observable board event rather than a symbol read from a textbook page [1]. Composing the 32-bit *microoperation* control word by hand requires packing nine bit fields into a fixed positional encoding before the value can be entered into program memory, disconnected from the register-transfer notation students already use; a small web-based utility built alongside this work, the Mano Datapath Micro-Operation Word Constructor, derives the 32-bit control word from a register-transfer expression such as R1 <- R2 + R3 and renders it as a ready-to-enter hexadecimal value; the VHDL source, testbenches, and this tool are available from the authors upon request. Table III maps three curriculum-topic groups [12], register-transfer and *ALU* microoperations, the fetch-decode-execute cycle, and the hierarchical datapath organisation, to specific demonstrations available through this workflow.

As shown in Table III, each topic maps to a specific switch setting, button sequence, and output signal on the Spartan-3AN board.

B. Discussion

The gap analysis in Section V changes what *fully implemented* means for the Mano model [1]. Read as a specification alone, the model appears self-contained; each engineering gap becomes visible only once an implementer attempts to place the design on a synthesis tool. *Fully implemented* therefore means resolved to the point a synthesis tool accepts and places the design, not merely transcribed into *VHDL* syntax.

Simulation-verified correctness and board-verified correctness are separate claims. The ENTRY RAM defect reported earlier in this section passed every pre-existing testbench yet

TABLE III
MAPPING OF COMPUTER ARCHITECTURE CURRICULUM TOPICS TO OBSERVABLE DEMONSTRATIONS ON THE SPARTAN-3AN DATAPATH BOARD.
SOURCE: [1], [12]

CA Topic	Board Demonstration
Register-transfer operations and *ALU* microoperations	MANUAL mode: BTN_NORTH executes one register-transfer step, *LED* bank LD[7:0] shows the updated register. AUTORUN mode: the same *LED*/*LCD* display each arithmetic/logic result in sequence.
Microoperation sequencing	Each BTN_NORTH press in MANUAL mode advances the sequencer by one 32-bit control word; the *LCD* mode line reports the current step index.
Hierarchical datapath organisation	Cycling register-file entries in READ REG mode with BTN_EAST/BTN_WEST shows each entry's independent value on the *LED* bank.

still misplaced a committed byte on the physical board; as established above, the cause was a coverage gap, not a limit of simulation, so passing an incomplete suite did not guarantee board-level correctness. The two regression testbenches added afterward close that gap, verified to fail against the pre-fix RTL and pass after. The distinction between suite coverage and board behaviour remains a property of the methodology, not of any single test.

VII. CONCLUSION

This work delivers three contributions: a systematic *gap analysis* resolving the engineering gaps between the Mano register-transfer specification [1] and a synthesisable hardware realisation; the resulting complete, hierarchically-structured VHDL-93 datapath, designed and implemented on the Spartan-3AN FPGA while preserving the single-cycle microoperation contract of the original model; and the functional-simulation and board-level verification of that datapath, reported in Section VI. Post-synthesis resource utilisation and timing closure, reported in Section V, confirm that the design meets the 50 MHz board-oscillator constraint.

Beyond answering the research question, the board realisation carries pedagogical value: it turns each register-transfer step and each microoperation of the Mano model into a directly observable change on physical output peripherals, rather than a symbolic notation to be reasoned about on paper or inferred from a software waveform trace. In the sense established in Section VI-B, this work confirms that the Mano register-transfer microoperation model can be implemented as a synthesisable, hierarchically-structured VHDL-93 design on an FPGA, while preserving its single-cycle microoperation contract and its textbook register-transfer decomposition.

A. Future Work

Three directions extend this work. The *microoperation sequencer* executes a flat, linear microprogram with no call or return mechanism; a dedicated call stack would let the *program counter* save its return address before branching into a reusable routine. The 8-bit word-width contraction adopted in Section V limits the register file and the addressable memory to 256 words; restoring the full 16-bit Mano word width would recover data parity with the original model and a 4,096-word address space. The pedagogical application in Section VI-A motivates a controlled study comparing board-based against lecture-only instruction.

REFERENCES

[1] M. M. Mano, *Computer System Architecture*, 3rd ed. Englewood Cliffs, NJ: Prentice-Hall, 1993.

[2] M. Holland, J. Harris, and S. Hauck, "Harnessing FPGAs for computer architecture education," in *Proceedings of the 2003 IEEE International Conference on Microelectronic Systems Education (MSE'03)*. Los Alamitos, CA: IEEE, 2003, pp. 1–2.

[3] V. Guštin and P. Bulić, "Learning computer architecture concepts with the FPGA-based "Move" microprocessor," *Computer Applications in Engineering Education*, vol. 14, no. 2, pp. 135–141, 2006.

[4] M. de los Ángeles Cifredo-Chacón, Á. Quirós-Olozábal, and J. M. Guerrero-Rodríguez, "Computer architecture and FPGAs: A learning-by-doing methodology for digital-native students," *Computer Applications in Engineering Education*, vol. 23, no. 4, pp. 464–470, 2015.

[5] M. M. Mano and C. R. Kime, *Logic and Computer Design Fundamentals*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2008.

[6] Xilinx, Inc., *Spartan-3AN FPGA Starter Kit Board User Guide*, San Jose, CA, 2009, no ISBN assigned; available at `xilinx.com`.

[7] ——, *Spartan-3AN FPGA Family Data Sheet*, San Jose, CA, 2019, no ISBN assigned; DS557 (v4.3); available at `xilinx.com`.

[8] ——, *ISE Design Suite 14: Release Notes, Installation, and Licensing*, San Jose, CA, 2012, no ISBN assigned; available at `xilinx.com`.

[9] V. A. Pedroni, *Circuit Design and Simulation with VHDL*, 2nd ed. Cambridge, MA: MIT Press, 2010.

[10] Xilinx, Inc., *ISE In-Depth Tutorial*, San Jose, CA, 2012, no ISBN assigned; UG695 (v14.1); available at `xilinx.com`.

[11] ——, *XST User Guide for Virtex-4, Virtex-5, Spartan-3, and Newer CPLD Devices*, San Jose, CA, 2013, no ISBN assigned; UG627 (v14.5); available at `xilinx.com`.

[12] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 10th ed. Hoboken, NJ: Pearson, 2016.